

プログラミング言語論

第2回

命令型言語

(構造化プログラミング、制御フロー)

篠埜 功

命令型言語について

命令型言語では、計算とはアクションの列であると見られる。

命令型言語の例：

Fortran (1957, John Backus, アメリカ人、1977 Turing賞),
Algol 60 (1960, 国際委員会で作成)

Pascal (1970, Niklaus Wirth, スイス人、1984 Turing賞),
C (1972, Dennis Ritchie, アメリカ人、1983 Turing賞)

Fortranではwhile文などがなく、goto文が多用され、制御フローがつかみにくい。(Fortran 90 (1991)では対応)

Algol, Pascalなどの言語では、プログラムの構造が明確になるような構成要素(while, begin endなど)が取り入れられた。

代入

命令型言語では、変数の値を変える操作が基本。
代入(assignment)によって変数の値を変える。

(代入の例)

$x := 2+3$

$x := a[i]$

$a[i] := x$

手続き呼び出しによって間接的に変数の値を変えることもできる。

(手続き呼び出しの例)

`read(x)`

構造化プログラミング (structured programming)

プログラムを見て、行われる計算が容易に分かるように言語を設計するべき。goto文を多用したプログラムではどのような計算が行われるのかわかりにくくなる。

構造化プログラミング: プログラムの構造が、どのような計算が行われるかの理解の助けになる。

構造化されたプログラムでも構造化されていないプログラムと同等の効率のものは書ける。

[参考文献] Edsger Dijkstra, “Go to statement considered harmful”, *Communication of the ACM*, Vol. 11, No. 3, pp. 147-148, 1968.

構造化とは

プログラムが構造化されている



プログラムの制御フローがプログラムテキストの構文構造から明らか

明らかというのは、ここではsingle entry, single exit(入口1つ、出口1つ)と定義する。

複合文(**begin-end**)

条件文(**if-then, if-then-else**)

繰り返し文(**while**)

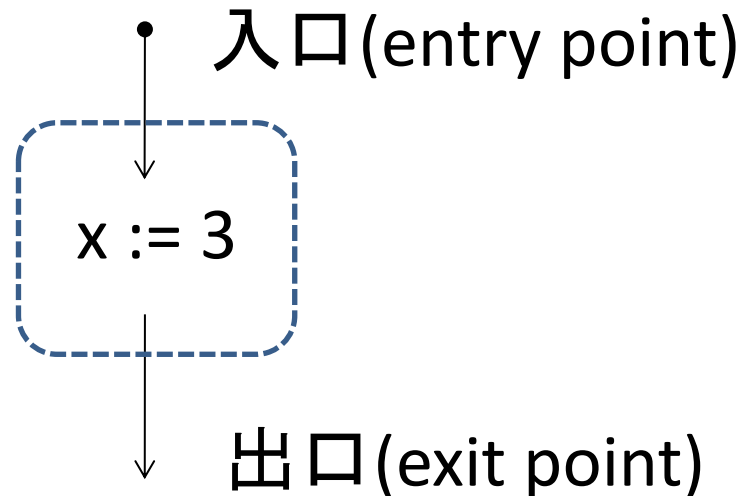
選択文(**case**)

基本ケース

代入文はsingle entry, single exitである。
たとえば、

$x := 3$

のような代入文の制御フローは、以下のように図示できる。



文の列 (逐次文)

Pascalなどでは、文の列 (逐次文) は文 $s_1, s_2, \dots, s_n$ をセミコロンで区切って表す。

$s_1; s_2; \dots; s_n$

(Cでは文を並べるだけでよい。)

(例) $\text{temp} := x;$

$x := y;$

$y := \text{temp}$

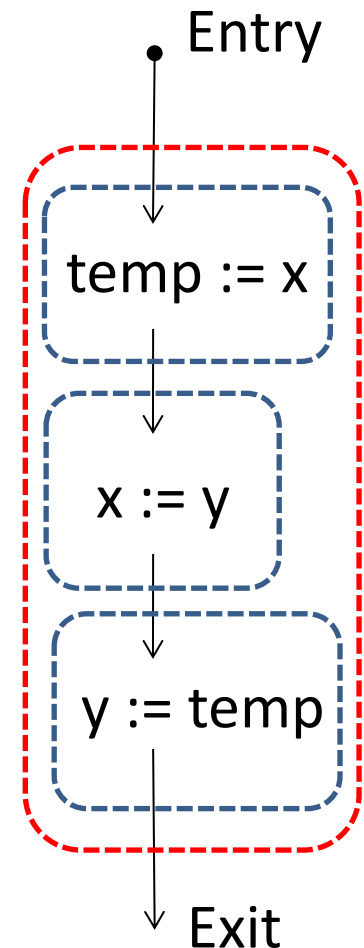
逐次文のグループ化（複合文、ブロック）

逐次文は、ALGOLやPascal等では**begin, end** (C言語では{と}) で囲むことにより、1つの複合文にまとめることができる。

(例) **begin** temp := x; x := y; y := temp **end**
複合文は、文が書けるところならどこにでも書くことができる。**begin, end**で囲む文は0個でもよく、

begin end

も複合文である。複合文は、それを構成する各文がsingle entry, single exitならsingle entry, single exitである。



条件文 (conditional statements)

条件文はPascal等では以下の形で書かれる。

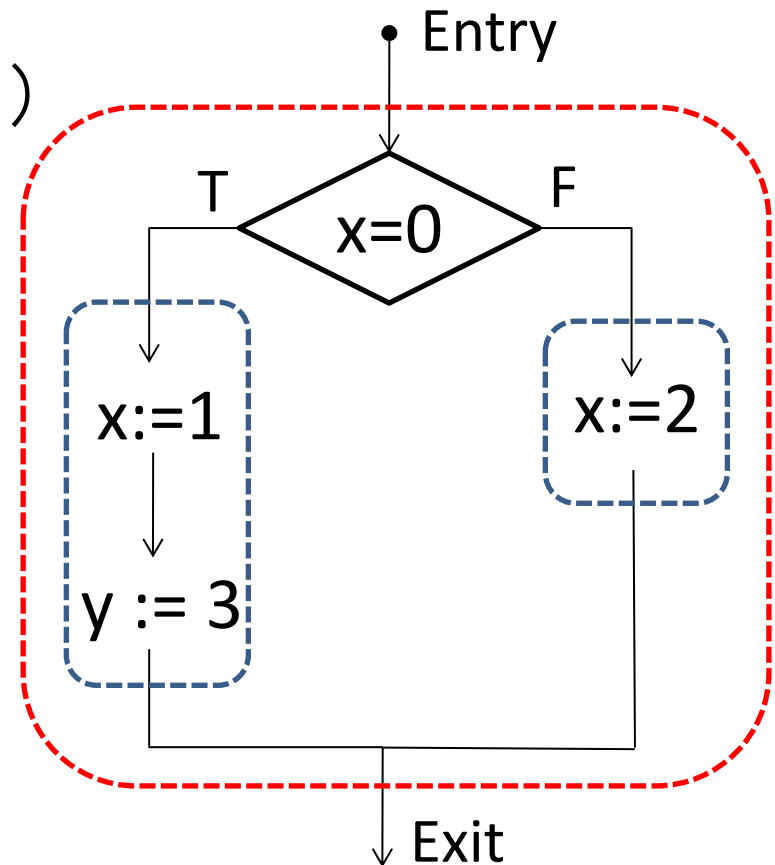
if E then S_1 else S_2

if E then S

(E は式、 S , S_1 , S_2 は文を表す。)

(例) **if $x=0$ then**
 begin $x:=1$; $y:=3$ end
 else $x:=2$

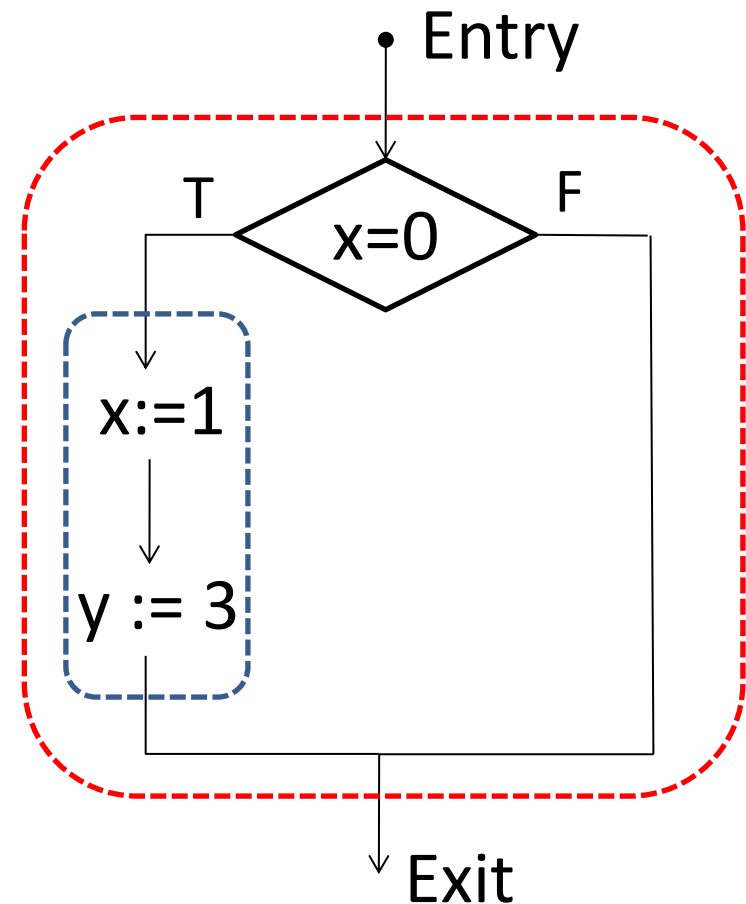
条件文 **if E then S_1 else S_2**
は、 S_1 と S_2 がsingle entry,
single exitならsingle entry,
single exitである。



elseノート無しの場合

(例) **if** $x=0$ **then**
 begin
 $x:=1$;
 $y:=3$
 end

elseノート無しのif文 **if** E
then S は、本体 S がsingle
entry / single exitなら
single entry / single exitで
ある。



練習問題1

以下のプログラム断片の制御フローを図示せよ。

if $x > 0$ **then** $x := x - 1$

else if $y > 0$ **then** $y := y - 1$

else $y := y + 1$

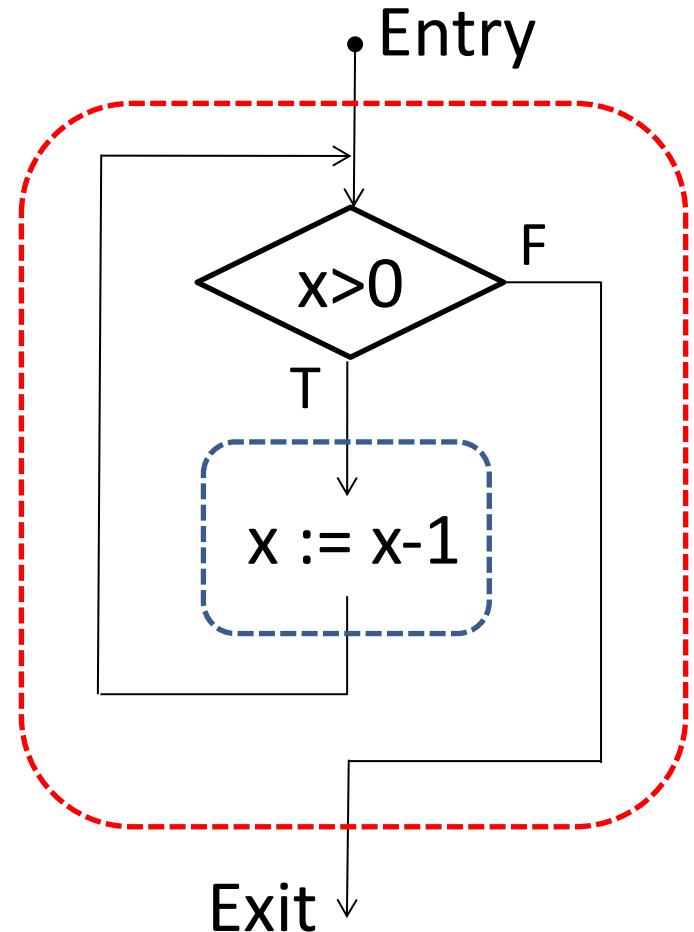
繰り返し文 (loop)

繰り返し文はPascal等では以下の形で書かれる。

while 式 do 文

(例) while $x > 0$ do $x := x - 1$

while文 **while E do S** は、
本体の文 S がsingle entry,
single exitならsingle entry,
single exitである。



選択文

選択文はPascal等では以下のような形で書かれる。

case 式 of

constant1 : 文1;

constant2 : 文2;

...

constantn : 文n

end

(Cでは選択文はswitch文)

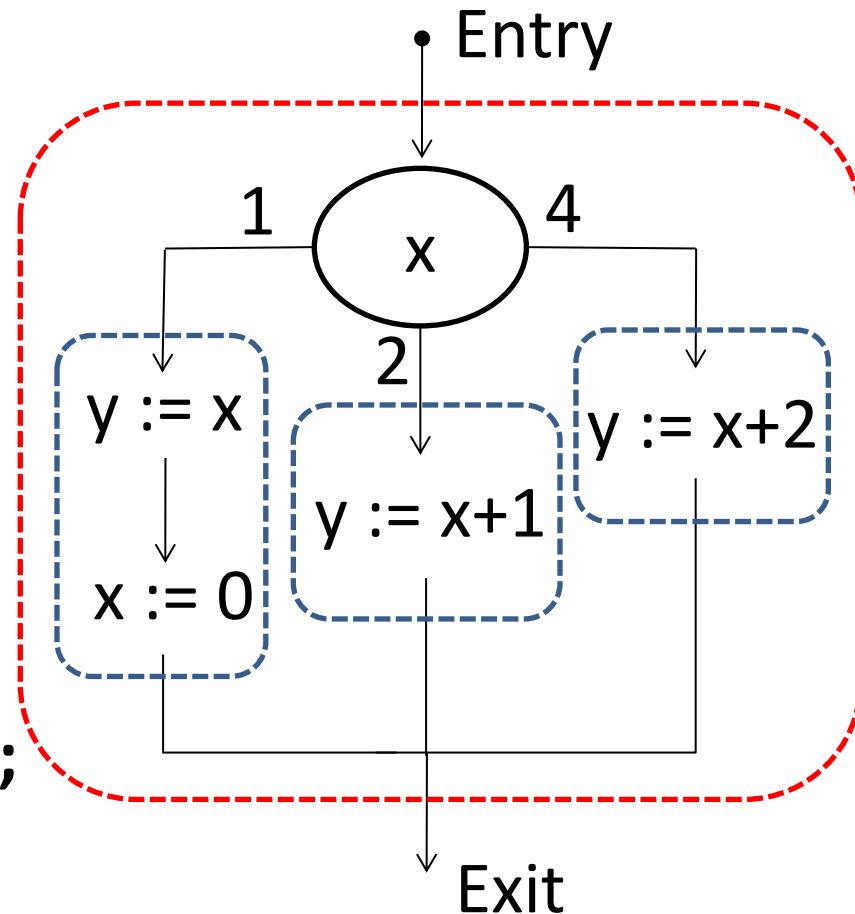
(例) **case x of**

1 : **begin** y:=x; x:=0 **end**;

2: y:=x+1;

4: y:=x+2

end



練習問題2

以下のプログラム断片の制御フローを図示せよ。

$y := 3;$

case x **of**

1 : $y := 1;$

2 : $y := x * 2;$

3 : **if** $z = 0$ **then**

$y := y * y$

else

$y := y * y * y$

end

繰り返し文における特殊ケースの扱い

break文、continue文(C言語等)

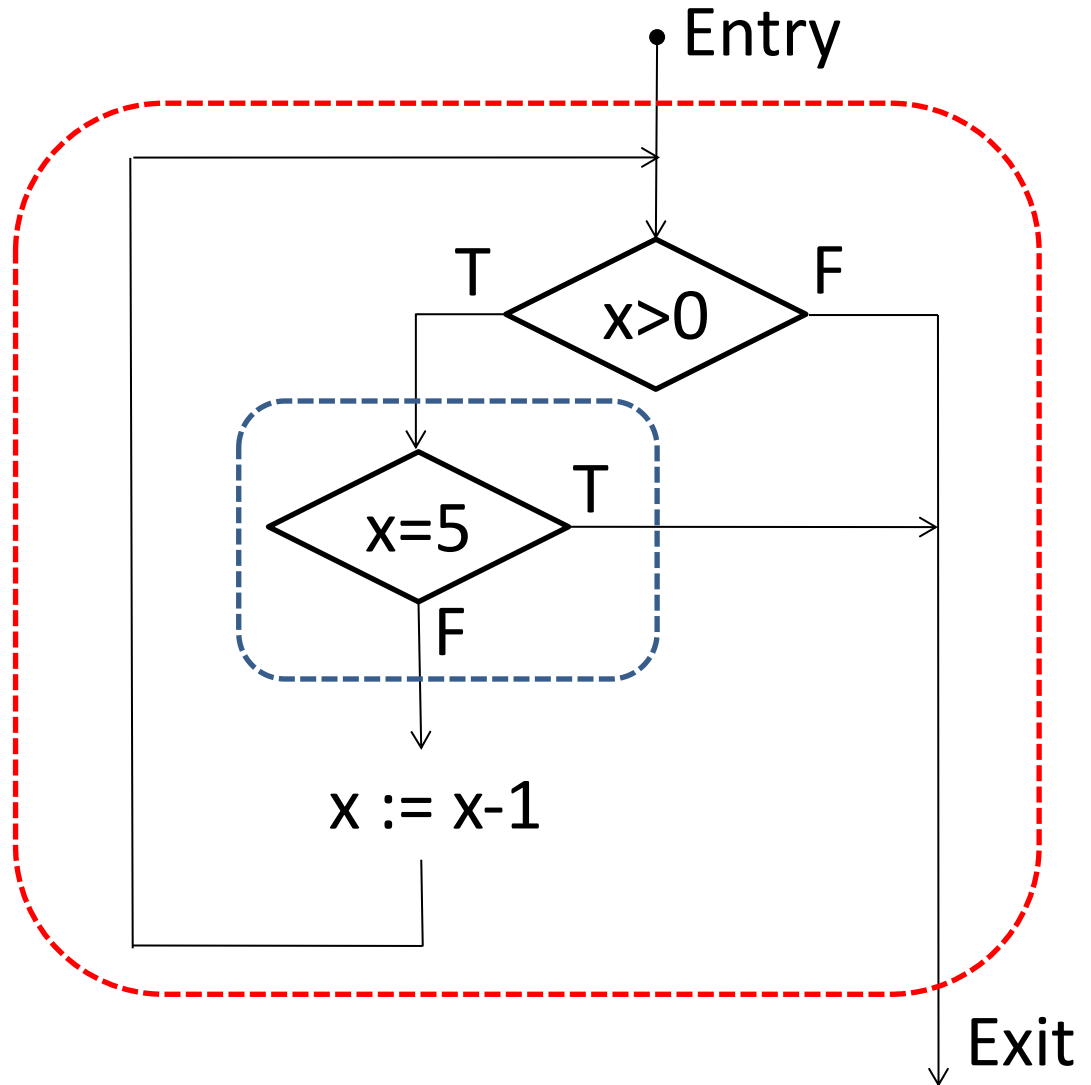
break文が実行されると、それが属する最も内側の繰り返し文を脱出する。(繰り返し文の次の文へ制御が移る。)

continue文が実行されると、それが属する最も内側の繰り返し文のループ継続部(ループ本体の終わり)に制御が移る。

break文の使用例

```
while x>0 do  
  begin  
    if x=5 then  
      break;  
    x := x-1  
  end
```

break文によって、
if文の出口は2つ
になったが、
while文全体は
single entry,
single exitである。



練習問題3

以下のプログラム断片の制御フローを図示せよ。

```
while x>0 do  
  begin  
    if x=3 then  
      break;  
    y := y + 1;  
    x := x - 1  
  end
```

練習問題4

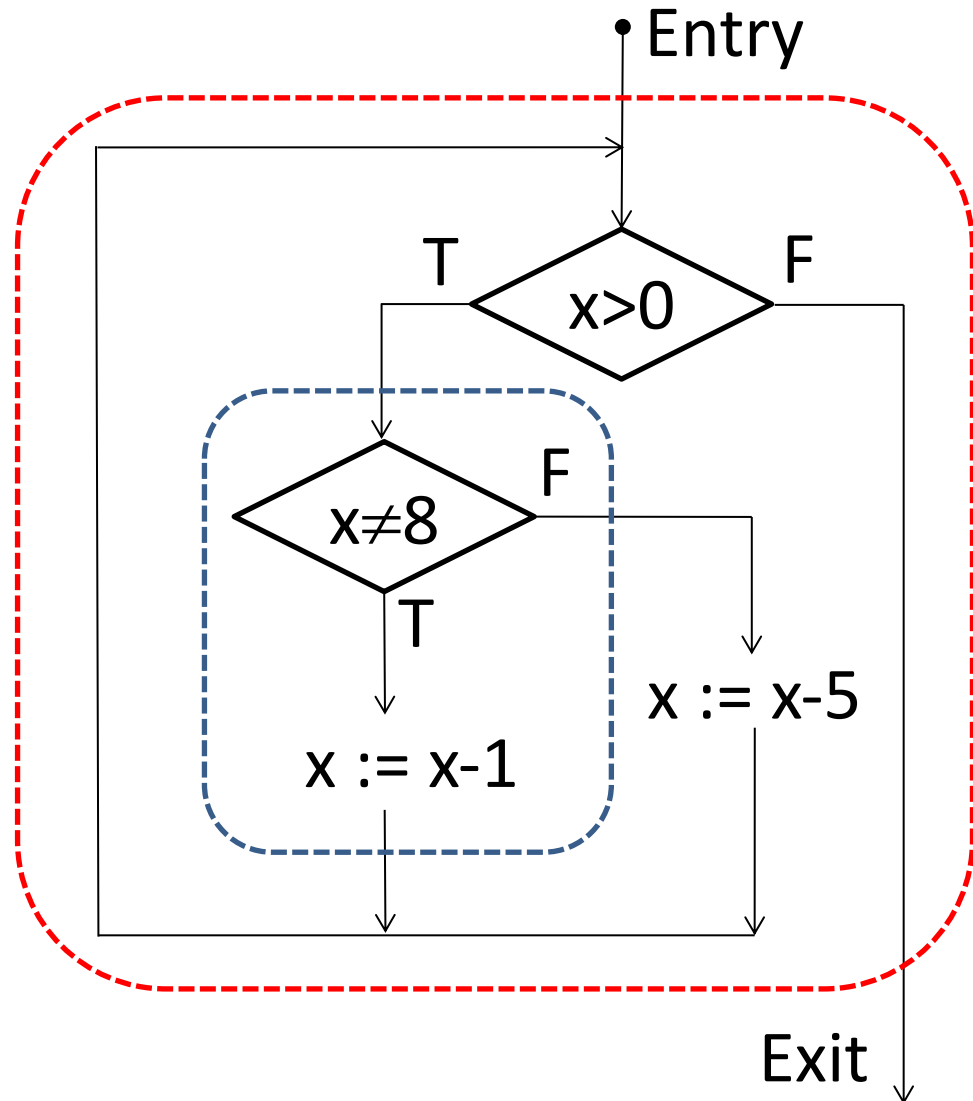
以下のプログラム断片の制御フローを図示せよ。

```
while x>0 do  
  begin  
    while y>0 do  
      begin  
        if x=3 then  
          break;  
        z := z + 1;  
        y := y - 1  
      end;  
    x := x - 1  
  end
```

continue文の使用例

```
while x>0 do
  begin
    if x ≠ 8 then
      begin
        x := x-1;
        continue
      end;
    x := x-5
  end
```

continue文によって、if文の出口は2つになったが、while文全体はsingle entry, single exitである。



練習問題5

以下のプログラム断片の制御フローを図示せよ。

```
while x>0 do  
  begin  
    while y>0 do  
      begin  
        if x≠3 then  
          begin  
            y := y - 1;  
            continue  
          end  
          z := z + 1; y := y - 1  
        end;  
      x := x - 1  
    end  
  end
```

goto文

goto文は、以下の
形式で書かれる。

goto ラベル

(例)

L: $x := x - 4;$

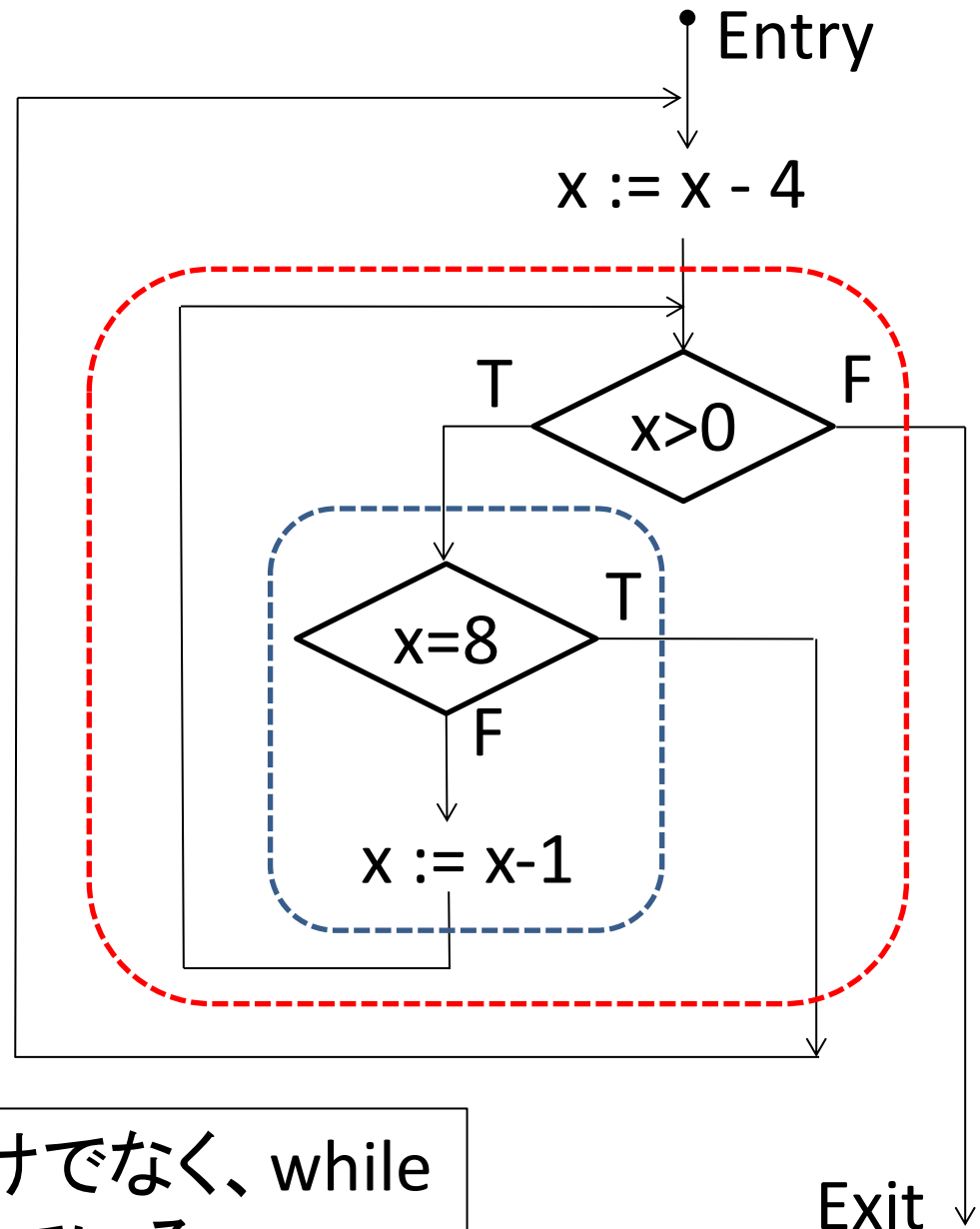
while $x > 0$ **do**

if $x = 8$ **then**

goto L

else

$x := x - 1$



goto文により、if文だけでなく、while
文の出口も2つになっている。

練習問題6

以下のプログラム断片の制御フローを図示せよ。

$x := 10;$

$\text{sum} := 0;$

L: $\text{sum} := \text{sum} + x;$

$x := x - 1;$

if $x > 0$ then

 goto L

練習問題7

以下のプログラム中の2か所のif文および内側のwhile文の入口、出口はそれぞれいくつか。

```
while x>0 do
  begin
    while y>0 do
      begin
        if x=3 then
          break;
        L: z := z + 1;
        y := y - 1
      end;
      x := x - 1;
      if x = 2 then goto L
    end
  end
```

return文

return文はModula-2などでは
return あるいは return 式
の形で書かれる(Cではセミコロンをつけてreturn;
あるいはreturn 式;と書く)。
return文が実行されると、その手続き(関数)を呼び
出した部分に制御が戻る。

Return文も、goto文、break文、continue文と同様、
実行されると制御が移る。

break文は繰り返しを脱出するのに対し、return文
は手続き(関数)を脱出する。